REACT · DERS MATERYALI

React Router DOM

SPA mantığı, route'lar, dinamik sayfalar ve yönlendirme

```
localhost:5173/ürünler/3
// URL'e karşılık gelen route
<Route
  path="/ürünler/:id"
  element={<Detail />}
/>
```

PLAN

Bugün neler öğreneceğiz?

01
SPA ve Router
Sayfa yenilemeden gezinme, kurulum

02
Temel yapı
Route, Link ve NavLink

03
Dinamik route
:id, useParams, optional chaining

04
İç içe route
Outlet, layout, yetkilendirme

05
Hook’lar ve 404
useNavigate, useSearchParams, useLocation

06
Modern tanım
createBrowserRouter ve RouterProvider

01

SPA ve React Router

Sayfa yenilenmeden, tek bir HTML dosyasıyla çok sayfalı bir site nasıl olur?

01 · SPA VE ROUTER

SPA (Single Page Application)

Geleneksel site

1. Bir linke tıklarsın
2. Tarayıcı sunucuya gider
3. Yeni bir HTML sayfası indirilir
4. Ekran baştan aşağı yeniden çizilir

SPA

1. Tarayıcıya yalnızca tek HTML yüklenir
2. Bir linke tıklarsın
3. JavaScript sadece değişen içeriği yeniler
4. Header ve footer yerinde kalır

Sonuç: sayfa yenilenmediği için geçişler hızlıdır ve değişmeyen bileşenlerin state'i korunur.

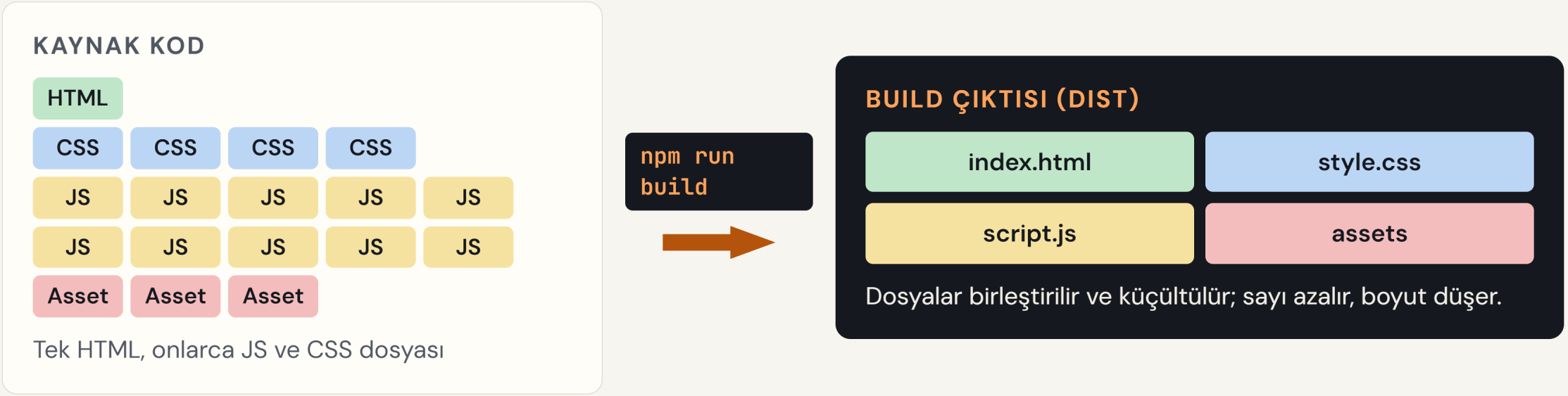
Geleneksel uygulama: her sayfada yeni dosyalar



HTML sayfa yapısı CSS stiller JS davranış

Dezavantaj: her sayfa geçişinde gerekli HTML, CSS ve JS dosyaları sıfırdan indirilir.

SPA: build (derleme) aşaması

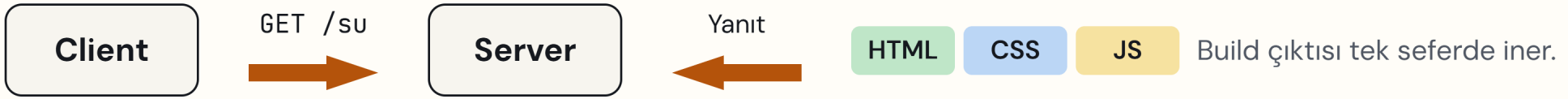


Build: projedeki dosyaların optimize edilip yayınlanmaya hazır hale getirilme süreci.

01 · SPA VE ROUTER

SPA: dosyalar bir kez iner

1 · İLK ZIYARET



2 · SONRAKI SAYFA GEÇIŞLERI



01 · SPA VE ROUTER

React Router DOM ne yapar?

Adres çubuğundaki URL'i dinler, sayfa yenilemesini durdurur ve URL'e karşılık gelen bileşeni ekrana basar.

1 · ADRES ÇUBUĞU

/ürünler

Kullanıcı bir linke tıklar, URL değişir

→

2 · ROUTER

Eşleştirir

Yenilemeyi durdurur, URL'e uyan route'u bulur

→

3 · EKRAN

<Products />

Karşılık gelen bileşen ekrana basılır

URL anatomisi

```
https://www.sahibinden.com:4000/otomobil/bmw/izmir?yil_min=2012&yil_max=2018
```

Parça	Örnek	React Router'da
Protocol	https://	Router'ın işi değil
Domain	www.sahibinden.com	Router'ın işi değil
Port	:4000	Router'ın işi değil
Path	/otomobil	Route path="/otomobil"
Path params	/bmw/izmir	:marka/:sehir ve useParams
Query params	?yil_min=2012&yil_max=2018	useSearchParams

01 · SPA VE ROUTER

Kurulum ve iki tanım yöntemi

```
# terminal
npm i react-router-dom

// kodda
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

Klasik tanım

BrowserRouter, Routes ve Route bileşenleriyle JSX içinde.

Modern tanım

createBrowserRouter ve RouterProvider ile nesne dizisi olarak. Son bölümde.

02

Temel yapı

BrowserRouter, Routes, Route, Link ve NavLink ile ilk sayfalarımızı kuralım.

02 · TEMEL YAPI

Üç temel bileşen

BrowserRouter

Tüm uygulamayı sarmalar; router'ın çalışmasını ve adres çubuğunu dinlemeyi sağlar.

Routes

Tüm route'ların kapsayıcısı; URL değişince hangi bileşenin basılacağını seçer.

Route

Her sayfa için bir tane; path (URL) ve element (bileşen) alır.

ÖRNEK: URL /ÜRÜNLER OLUNCA PRODUCTS BILEŞENINI BAS

```
<Route path="/ürünler" element={<Products />} />
```

Klasik yapı: App.jsx

```
<BrowserRouter>
  <Header />

  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/ürünler" element={<Products />} />
    <Route path="/hakkımızda" element={<About />} />
  </Routes>

  <Footer />
</BrowserRouter>
```

Header ve Footer Routes'un dışında: her sayfada sabit kalır.

path URL'i, **element** basılacak bileşeni belirtir.

Adreste Türkçe karakter çalışır (/ürünler); pratikte çoğu zaman ASCII tercih edilir.

02 · TEMEL YAPI

Link: sayfayı yenilemeden gezinme

<a href="/ürünler">

Tarayıcı sunucuya gider; sayfa baştan yüklenir ve state sıfırlanır.

<Link to="/ürünler">

Sayfa yenilenmez; yalnızca URL değişir ve ilgili bileşen basılır.

CARD.JSX · HREF YERINE TO PROP'U

```
<Link to={`/ürünler/${book.id}`}>
  Detay'a Git
</Link>
```

NavLink: aktif sayfayı işaretleme

```jsx
// Header.jsx
<nav>
  <NavLink to="/">Anasayfa</NavLink>
  <NavLink to="/ürünler">Ürünler</NavLink>
  <NavLink to="/kategori/roman">Roman</NavLink>
</nav>
```

```jsx
// class'ı kendin belirlemek istersen
<NavLink
  to="/ürünler"
  className={({ isActive }) ⇒ (isActive ? "aktif" : "")}
>
```

Link ile aynı işi yapar, **to** prop'u alır.

Ziyaret edilen sayfanın linkine otomatik **active** class'ı verir.

CSS'te **nav a.active** ile stil verilir.

03

Dinamik route

Tek bir sayfa şablonu, URL’deki parametreye göre değişen içerik.

03 · DINAMIK ROUTE

Bir route, birçok URL

/ürünler/1

/ürünler/2

/ürünler/3



```
<Route
  path="/ürünler/:id"
  element={<Detail />}
/>
```

:id bir parametredir; değeri URL'den gelir (1, 2, 3…).

Detay sayfaları gibi içeriği URL'e göre değişen sayfalarda kullanılır.

useParams: URL'deki parametreye erişim

```jsx
// Detail.jsx
const { id } = useParams();

useEffect(() => {
  api
    .get(`/books/${id}`)
    .then((res) => setBook(res.data));
}, [id]);
```

useParams, path'teki **:id** gibi parametreleri nesne olarak verir.

Değer her zaman **string** gelir ("3" gibi).

Bağımlılık dizisine **id** yazılır; URL değişince yeni kitap istenir.

03 · DINAMIK ROUTE

Detay sayfası akışı

1	Kullanıcı karta tıklar	`<Link to="/ürünler/3">`
2	URL değişir, route eşleşir	`/ürünler/:id`
3	Detail bileşeni ekrana basılır	`<Detail />`
4	id parametresi okunur	`const { id } = useParams()`
5	API'dan kitap alınır ve gösterilir	`GET /books/3`

03 · DINAMIK ROUTE

Optional chaining (?.)

HATA: API YANITI GELMEDEN ERIŞIM

```
const [book, setBook] = useState(null);

<h1>{book.title}</h1>
// TypeError: Cannot read
// properties of null
```

ÇÖZÜM: OPTIONAL CHAINING

```
const [book, setBook] = useState(null);

<h1>{book?.title}</h1>
// book null ise undefined döner,
// hata oluşmaz
```

?. nesne varsa özelliğe eriş, yoksa undefined döndür.

Alternatif: **isLoading** iken erken return ile bileşeni durdur (Detail böyle yapıyor).

04

İç içe route

Ortak arayüzü ve ortak kontrolleri tek bir yerde toplayalım.

04 · İÇ IÇE ROUTE

Nested route ve Outlet

APP.JSX · KAPSAYICI VE ALT ROUTE'LAR

```
<Route path="/kategori" element={<Category />}>
  <Route path="roman" element={<Novel />} />
  <Route path="hikaye" element={<Story />} />
</Route>
```

CATEGORY.JSX · ALT ROUTE'UN YERI

```
const Category = () => (
  <div>
    <aside>aside alanı</aside>
    <Outlet />
  </div>
);
```

URL	Ekrana basılan
/kategori/roman	Category içinde Novel
/kategori/hikaye	Category içinde Story

04 · İÇ IÇE ROUTE

Layout route: ortak arayüz

LAYOUT.JSX

```
<div>
  <Header />
  <main>
    <Outlet />
  </main>
  <Footer />
</div>
```

APP.JSX · PATH'I OLMAYAN ROUTE

```
<Route element={<Layout />}>
  <Route path="/" element={<Home />} />
  <Route path="/ürünler" element={<Products />} />
</Route>
```

Path'i olmayan route yalnızca sarmalayıcıdır; URL'e bir şey eklemez.

Header ve Footer tek yerde durur; her sayfaya yeniden yazılmaz.

04 · İÇ IÇE ROUTE

Ortak yetkilendirme kontrolü

```jsx
// Protected.jsx
const Protected = ({ isAdmin }) => {
  if (!isAdmin) return <Navigate to="/" replace />;

  return <Outlet />;
};
```

```jsx
<Route element={<Protected isAdmin={isAdmin} />}>
  <Route path="/kategori" element={<Category />} />
</Route>
```

Rol veya oturum kontrolü **tek yerde** yapılır; korunan sayfalar bu route'un çocuğu olur.

Navigate: useNavigate'in bileşen hali; render sırasında yönlendirir. replace geçmişe eklemez.

Örnek projede bu fikir Category.jsx içinde yorum satırı olarak duruyor.

05

Hook’lar ve 404

Kodla yönlendirme, arama parametreleri, taşınan veri ve bulunamayan sayfa.

useNavigate: kodla yönlendirme

```
const navigate = useNavigate();

navigate("/ürünler");            // sayfaya git
navigate(-1);                  // bir sayfa geri
navigate("/", { replace: true });
// replace: geçmişe yeni kayıt ekleme
navigate("/404", { state: msg });
// state: yanında veri taşı
```

Link'in **fonksiyon** versiyonudur.

Link: kullanıcı tıklar. **useNavigate:** kod karar verir (istek bitince, form gönderilince).

Detail'de API hata verirse kullanıcı /404'e yönlendirilir.

useSearchParams: arama parametreleri

```
const [searchParams, setSearchParams] = useSearchParams();
const searchTerm = searchParams.get("aramaTerimi");

useEffect(() => {
  api
    .get("/books", { params: { q: searchTerm } })
    .then((res) => setBooks(res.data));
}, [searchTerm]);

// input onChange içinde:
searchParams.set("aramaTerimi", e.target.value);
setSearchParams(searchParams);
```

URL'de **?** işaretinden sonra gelen **key=value** çiftleri.

Arama URL'de durur: sayfa yenilense de korunur, link paylaşılabilir.

searchTerm bağımlılıkta: URL değişince API isteği yenilenir.

useLocation: URL bilgisi ve taşınan veri

```
// Detail.jsx · hata olunca yönlendir
navigate("/404", { state: err.message });

// NotFound.jsx
const location = useLocation();

<h1>
  {location.state ? location.state : "SAYFA BULUNAMADI"}
</h1>
```

location nesnesi

- pathname: /ürünler
- search: ?aramaTerimi=roman
- hash: #bolum
- state: taşınan veri

404 sayfası: path="*"

```
<Route path="*" element={<NotFound />} />
```

Geçersiz URL

/abc gibi hiçbir route ile eşleşmeyen adresler.

Yıldız (*)

Her adresle eşleşir; başka route bulunamazsa çalışır.

API hatası

Detail, navigate("/404") ile bu sayfaya yönlendirir.

06

Modern tanım

createBrowserRouter ile aynı yapıyı nesne dizisi olarak yazalım.

06 · MODERN TANIM

createBrowserRouter

```
const router = createBrowserRouter([
  {
    element: <Layout />,
    children: [
      { path: "/", element: <Home /> },
      { path: "/ürünler/:id", element: <Detail /> },
      {
        path: "/kategori",
        element: <Category />,
        children: [{ path: "roman", element: <Novel /> }],
      },
      { path: "*", element: <NotFound /> },
    ],
  },
]);
<RouterProvider router={router} />
```

Route'lar **nesne dizisi** olarak tanımlanır: path, element, children.

RouterProvider router'ı uygulamaya bağlar; BrowserRouter'ın yerini alır.

path'i olmayan nesne layout route'tur; children'ı sarar.

Klasik ve modern tanım

	Klasik	Modern
Tanım	JSX: Route bileşenleri	Nesne dizisi
Sarmalayıcı	BrowserRouter	RouterProvider
İç içe route	Route içinde Route	children dizisi
Ek özellik	Yok	loader, action gibi veri API'ları

İkisi de aynı sayfaları üretir; modern tanım ek özelliklere kapı açar.

06 · MODERN TANIM

Router hook’ları bir arada

Hook	Ne işe yarar?	Projede
useParams	Path’teki :id gibi parametreleri okur	Detail.jsx
useSearchParams	URL’deki ?key=value değerlerini okur ve yazar	Products.jsx
useNavigate	Kodla yönlendirme yapar	Detail.jsx
useLocation	URL bilgisini ve taşınan state’i verir	NotFound.jsx

DIKKAT

Sık yapılan hatalar

a etiketiyle gezinmek

Sayfa yenilenir, state kaybolur. Uygulama içinde Link kullan.

Link to={-1}

Link sayı kabul etmez. Geri gitmek için navigate(-1) kullan.

Boş bağımlılık dizisi

Detail'de [ ] varsa id değişince istek atılmaz. [id] yaz.

Outlet unutmak

Kapsayıcı route'ta Outlet yoksa alt route görünmez.

Parametre adı farklı

path'te :id ise useParams'ta da id yaz; yoksa undefined gelir.

Hata ayıklarken

useLocation() çıktısına bak: pathname URL ile eşleşiyor mu?

ÖZET

Bugün öğrendiklerimiz

Temel

BrowserRouter, Routes, Route. Link ve NavLink sayfayı yenilemeden gezdirir.

Dinamik

:id ile tek route, useParams ile okuma, ?. ile güvenli erişim.

İç içe

Outlet ile alt route'lar. Layout ve yetkilendirme tek yerde.

Hook'lar

useNavigate, useSearchParams, useLocation. path="*" ile 404.

ALIŞTIRMA

Sıra sizde

1

Yeni alt route

/kategori altına gerilim route'unu ekle, Header'a NavLink koy.

2

Kategori filtresi

?kategori=roman parametresiyle listeyi filtrele (useSearchParams ve params).

3

Korumalı route

Admin olmayanlar /kategori'ye girmesin; Navigate ile ana sayfaya yönlendir.

Bonus: Detail'deki Geri linkini navigate(-1) kullanan bir butona çevir.